

MASTER PRODUCT REQUIREMENTS DOCUMENT

Chennai Traffic Intelligence & Multidimensional Visualization Platform

For Advanced Data Visualization Techniques

Master product specification for Antigravity development

Product principle: The application is an interactive traffic-authority intelligence product, not a collection of static charts. The Chennai map is the primary interface; every other analysis is reached through interaction with the map, filters, time controls, and drill-down.

1. Executive Product Definition

Build a web-based traffic intelligence platform for traffic authorities and traffic inspectors operating in Chennai. When a user enters the system, the first and dominant experience is an interactive Chennai road map showing congestion geographically.

Congested roads are visually encoded by severity: Low, Moderate, High, and Severe. Accident information is represented as a separate visual layer so that a user can distinguish congestion from incidents. Locations with both severe congestion and significant accident activity receive a stronger priority/highlight treatment.

When an inspector clicks a road, junction, or hotspot, a location intelligence panel opens. The panel summarizes the selected location and provides deeper visual analysis: traffic trends, speed/volume relationships, accident patterns, weather associations when available, geographic context, traffic flow, and later ML predictions.

The product's central analytical story is: **Where** → **When** → **What is happening** → **What factors are associated** → **What is likely next** → **What needs attention**.

2. Primary User

Primary user: traffic authority / traffic inspector / traffic analyst. The product should optimize for fast geographic understanding and drill-down rather than requiring the user to navigate through many unrelated dashboards.

The interface should support an authority user who begins with the city-wide situation, identifies a problem location, clicks it, and immediately receives evidence about that location.

3. Product Goals

Create a professional interactive Chennai traffic map as the primary entry point.

Show congestion severity spatially on roads and locations.

Show accidents as a distinct layer and identify high-priority locations.

Allow click-to-investigate drill-down from city → area → road/junction → detailed location.

Provide advanced visualization techniques through meaningful interaction.

Support historical exploration using date/time filters and time animation.

Provide a clean interface for a separately developed ML prediction module.

Ensure observed, derived, simulated, and predicted information are never confused.

4. Non-Goals / Boundaries

The product does not autonomously control traffic signals.

It is a decision-support and visualization system.

ML model development is owned separately by the ML workstream; this product must provide a clean integration contract.

Do not claim live traffic unless the connected data source is genuinely live.

Do not present correlation as causation.

Do not present synthetic/simulated data as real observations.

Do not invent accident, weather, or traffic values merely to make the UI look complete.

5. Core User Journey

Step 1: User opens website → sees Chennai map.

Step 2: Map loads traffic condition layers and key indicators.

Step 3: User filters by date/time/area/condition as needed.

Step 4: User visually identifies red/severe areas and accident markers.

Step 5: User clicks a road/hotspot.

Step 6: Location Intelligence Panel opens without losing map context.

Step 7: User sees quick KPIs, then detailed visualizations.

Step 8: User investigates temporal, multivariate, accident, weather, and flow patterns.

Step 9: User optionally views ML prediction from the separate ML service/output.

Step 10: System summarizes priority/attention information.

6. Information Architecture

Primary navigation should remain simple. Recommended top-level areas: **Traffic Map**, **Location Analysis**, **Traffic Flow**, **Insights**, and **Prediction**. The map remains persistent or easily recoverable.

Do not make users hunt through ten separate chart pages. Location-specific analysis should be accessible directly from a selected map feature.

7. Home / Traffic Map

The map is the hero component and should occupy most of the first screen.

Required map elements: Chennai road network, congestion color encoding, accident markers, selected-location state, legend, map controls, current filters, and optional priority markers.

Congestion color semantics: Low = green, Moderate = yellow, High = orange, Severe = red. Use icons, labels, patterns, or tooltips in addition to color for accessibility.

Accidents must use a separate icon/marker layer. If accident severity or frequency is available, marker size or a secondary visual encoding can communicate magnitude.

A location with severe congestion plus repeated/significant accidents can receive a priority ring/glow/badge, but this must be driven by documented rules rather than visual guesswork.

8. Map Interaction Model

Hover: show compact tooltip with road/location, congestion, speed, volume, and incident summary where available.

Click: select road/junction/location and open Location Intelligence Panel.

Zoom: progressively reveal more detailed road/location information.

Filter: update map and all compatible analytics consistently.

Time slider: update the map state for the selected timestamp/period.

Layer toggle: Congestion / Accidents / Traffic Flow / Priority / Prediction.

Reset: restore city-wide view and default filter state.

9. Location Intelligence Panel

The panel is the most important secondary component. It should open as a right-side drawer, modal, or responsive detail panel while retaining the map context.

Header: location/road name, current severity, priority status, timestamp/filter context.

Quick metrics: vehicle count, average speed, density/utilization if available, accident count, weather condition if available, road capacity if available.

Detailed visual sections: traffic trend, congestion trend, vehicle count vs speed, accident-by-time chart, weather association, local mini-map, traffic-flow view, and prediction when available.

Include a clear 'Back to city' or close action. Avoid overwhelming the user with every chart at once; use sections/tabs/accordion where appropriate.

10. Advanced Visualization Requirements

Interactive geospatial visualization: road-level congestion map.

Traffic heatmap: density/congestion intensity.

Time-series visualization: traffic volume, speed, congestion.

Day × hour heatmap: recurring temporal patterns.

Multivariate comparison: compare selected traffic variables.

Scatter plots: vehicle count vs speed and other available variables.

Correlation visualization: association matrix with careful interpretation.

Flow/trajectory visualization: directional flow or actual trajectories if data supports them.

Geographic clustering: cluster locations based on traffic behavior.

Time-based animation: show congestion evolution.

Predictive visualization: future congestion/traffic state from ML output.

Ranking visualization: top congested roads, hotspots, or priority locations.

11. Time Interaction

Provide a date selector and time range where the data supports it.

Provide a time slider for supported historical periods. Moving the slider should update the map and relevant charts.

Provide play/pause animation when practical. The UI should display the exact timestamp being shown.

Never imply real-time playback when the underlying dataset is historical.

12. Accident Visualization

Accidents are a distinct visual layer from congestion.

Show accident count and severity for selected locations where available.

Show accident frequency by hour/day/date.

Show accident hotspots on the map.

Allow filtering or toggling accident layers.

Use 'risk indicator' terminology unless a validated accident-risk model exists.

13. Traffic Flow Visualization

If true trajectory/origin-destination data exists, visualize direction, volume, and movement between locations.

If only aggregate direction data exists, use directional flow arrows/lines scaled to traffic volume.

If flow is simulated for UI development, explicitly label it 'Simulated Flow' and keep it replaceable by real trajectory data later.

14. ML Integration Boundary

The ML workstream is separate. The dashboard must not depend on knowing how the model was trained.

Define an integration contract that can consume a file, database table, or API response.

Minimum prediction payload should include: location_id/road_id, prediction_timestamp, horizon, predicted_congestion, and confidence/probability where available.

Preferred advanced payload can additionally include predicted_vehicle_count, predicted_speed, model_version, top_features, and explanation data.

The dashboard should visually distinguish **Observed**, **Derived**, **Predicted**, and **Simulated** states.

Example prediction display: 'Predicted severe congestion in +30 min' with model confidence if the ML team supplies a calibrated confidence/probability.

15. Data Architecture

Use a canonical internal schema so multiple Chennai data sources can be integrated without rewriting the UI.

Potential sources: traffic/mobility, trajectory, weather, accidents, road network, and vehicle-registration/context data.

Real Chennai data should be preferred. Synthetic data is permitted only for development gaps and must be explicitly labeled.

Every source should have metadata: source name, period, spatial resolution, temporal resolution, available fields, license/access information, and limitations.

16. Canonical Data Fields

Identity: record_id, timestamp.

Location: latitude, longitude, road_name, road_id, junction, zone, road_type.

Traffic: vehicle_count, average_speed, traffic_density, vehicle_type, direction.

Time: date, hour, day_of_week, weekend_flag, peak_hour_flag.

Weather: weather_condition, rainfall, temperature, visibility.

Road: road_condition, road_capacity, lane_count, speed_limit.

Accident: accident_count, accident_severity.

Derived: traffic_utilization, speed_reduction, congestion_index, congestion_level.

Prediction: prediction_timestamp, horizon, predicted_congestion, confidence, predicted_speed, predicted_vehicle_count.

17. Congestion Representation

Implement a documented and configurable congestion index if raw congestion labels are unavailable. A baseline can combine traffic utilization and normalized speed reduction; additional contextual variables may be used when justified.

Store the formula and thresholds in configuration rather than burying them inside UI code.

All map colors, KPI labels, rankings, and filters should use the same canonical congestion definition.

18. Insights / Decision Support

Generate data-driven observations such as peak periods, persistent hotspots, high traffic/low speed associations, accident concentration, and predicted severe locations.

Insights must be computed from the active dataset/filter state. Never hard-code example findings as if they were real.

Priority levels can be Low / Medium / High / Critical and should be based on documented factors such as congestion severity, persistence, traffic volume, incidents, and prediction.

The product can recommend attention to a location, but it must not issue autonomous operational commands.

19. Recommended Technical Architecture

Frontend/application: React/Next.js or equivalent modern web stack if a true website experience is desired; use a Python API/service layer for analytics where appropriate.

Backend: Python FastAPI is recommended for data/analytics endpoints if frontend and backend are separated.

Data: Pandas/Polars for processing; Parquet for analytical files; PostgreSQL/PostGIS as a future scalable spatial database.

Visualization: Plotly for analytical charts; MapLibre GL JS / Leaflet / Mapbox-compatible approach for map rendering depending on project constraints.

ML integration: REST API or prediction artifact contract. Do not couple the UI to model internals.

The architecture must remain modular so data source, map engine, and ML implementation can evolve independently.

20. Suggested Repository Structure

traffic-intelligence-platform/

■ ■ ■ ■ frontend/

■ ■ ■ ■ components/

■ ■ ■ ■ pages/ or app/

■ ■ ■ ■ map/

■ ■ ■ ■ charts/

■ ■ ■ ■ styles/

■ ■ ■ ■ backend/

■ ■ ■ ■ api/

■ ■ ■ ■ services/

■ ■ ■ ■ analytics/

■ ■ ■ ■ data/

■ ■ ■ ■ models/

■ ■ ■ ■ data/

■ ■ ■ ■ raw/

■ ■ ■ ■ processed/

■ ■ ■ ■ synthetic/

■ ■ ■ ■ ml-contract/

■ ■ ■ ■ tests/

■ ■ ■ ■ docs/

■ ■ ■ ■ README.md

21. Development Roadmap

Phase 0 — Product foundation: PRD, architecture, UI wireframe, data contract.

Phase 1 — Data foundation: real Chennai data adapters, canonical schema, preprocessing, data-quality report.

Phase 2 — Map MVP: Chennai road map, congestion layer, accident layer, legend, hover/click interaction.

Phase 3 — Location Intelligence: right-side panel, KPIs, road details, local mini-map.

Phase 4 — Advanced visualization: time series, heatmaps, scatter plots, correlation, rankings.

Phase 5 — Interaction: coordinated filters, time slider, animation, drill-down.

Phase 6 — Flow and clustering: directional flow, trajectories if available, geographic clusters.

Phase 7 — ML integration: consume separately produced predictions and explanations.

Phase 8 — Insights: automatic data-driven insights and priority areas.

Phase 9 — Production hardening: tests, performance, responsive UI, error states, documentation.

Phase 10 — Final demo: end-to-end Chennai scenario demonstrating city map → click hotspot → deep analysis → prediction → priority.

22. Review Roadmap

Review 1: demonstrate the product concept, architecture, Chennai map, congestion visualization, accident layer, basic click interaction, sample/real data, and initial visual analytics.

Review 2: demonstrate full interactive analysis: temporal animation, heatmaps, scatter/correlation views, traffic flow, geographic clustering, and coordinated filters.

Review 3: demonstrate ML integration: prediction map, future time series, model result interpretation, and confidence/feature information supplied by ML team.

Final: demonstrate complete traffic-authority workflow from city-wide monitoring to location investigation and predictive decision support.

23. UX Rules

Map-first design. Do not make the homepage a grid of unrelated charts.

Use a restrained professional traffic-control visual language.

Maintain strong hierarchy: map → priority → selected location → detailed evidence.

Use tooltips and units everywhere necessary.

Provide loading, empty, error, and unavailable-data states.

Make selected road/location visually obvious.

Keep the current filter context visible.

Do not use color as the only communication mechanism.

Optimize for desktop/laptop traffic-control use first; ensure reasonable responsive behavior.

24. Performance & Reliability

Do not load huge raw datasets directly into the browser.

Pre-aggregate or query spatial/temporal data efficiently.

Cache expensive analytical operations.

Keep map rendering responsive.

Gracefully handle missing fields and empty filter results.

Log data/API errors without exposing sensitive implementation details to end users.

25. Testing

Unit tests for data normalization, congestion calculation, filters, insight generation, and ML payload validation.

Integration tests for map selection → location panel flow.

API contract tests for prediction payloads.

UI tests for core interactions where tooling permits.

Test empty datasets, missing weather/accident fields, invalid coordinates, and unavailable predictions.

26. Acceptance Criteria

The first screen clearly shows Chennai geography and congestion state.

Roads/locations are colored consistently according to congestion severity.

Accidents appear as a separate, understandable visual layer.

Clicking a road/location opens detailed intelligence without losing context.

The detail panel contains meaningful traffic metrics and multiple advanced visualizations.

Filters update all compatible components consistently.

Time slider/animation changes historical map state where data supports it.

Flow and clustering are clearly tied to real or explicitly simulated data.

ML predictions can be plugged in without rewriting the dashboard.

Observed vs predicted vs simulated information is clearly distinguished.

Insights are calculated from data, not hard-coded.

The complete application supports the workflow: city overview → identify hotspot → investigate → understand patterns → view prediction → prioritize attention.

27. Senior Developer Implementation Rules for Antigravity

Treat this document as the **source of truth** for product behavior and architecture.

Before implementing, inspect the repository and existing files. Do not destroy working work without understanding it.

Do not create a disposable demo if a production-oriented component can be created directly.

Implement in small, testable increments and keep the application runnable after each milestone.

Do not invent data columns or external APIs. Inspect actual datasets before coding source-specific transformations.

If a requested data dimension is unavailable, build a clean optional adapter and make the UI show 'Data unavailable' rather than fabricating values.

Do not hard-code Chennai road geometry. Use a proper road/geospatial source or an explicitly documented development fixture.

Keep configuration, data processing, analytics, UI, and ML integration separate.

Document every major architectural decision in docs/architecture.md.

Maintain a docs/progress.md file recording completed milestones, current blockers, assumptions, and next steps.

Maintain a docs/data-contract.md file describing canonical fields and source mappings.

Maintain a docs/ml-integration.md file describing the prediction payload contract without implementing the ML model itself.

28. Antigravity Execution Prompt

After reading this PRD, do the following in order:

1. Inspect the current repository/workspace.
2. Summarize the existing state and identify what is already implemented.
3. Create or update the project architecture to match this PRD.
4. Create the canonical data contract and sample development schema.
5. Build the map-first application shell.
6. Implement Chennai map rendering with a replaceable geospatial data layer.
7. Implement congestion and accident layers with clear legends.
8. Implement click-to-open Location Intelligence Panel.
9. Implement the first set of location-level visualizations.
10. Add filters and time interaction.
11. Add flow, clustering, insights, and ML integration only after the core interaction is stable.
12. Test each milestone and document it.

Do not stop at a static mockup. Build the functional architecture needed for the complete end product.

If real data is not yet connected, use a clearly isolated development fixture and make the replacement path obvious.

The most important acceptance test is: **open website → see Chennai map → identify red congestion → see accident highlight → click location → receive detailed intelligence panel → explore visual analysis → receive ML prediction when connected.**

29. Example End-to-End Scenario

A traffic inspector opens the system at 08:30. The Chennai map shows several severe congestion segments in red. One location also has a prominent accident marker and is ranked as a priority area.

The inspector clicks the location. The right-side Location Intelligence Panel opens and shows: severe congestion, vehicle count, average speed, accident count, current filter context, and the selected road on a local map.

The inspector scrolls through the panel. A time-series chart shows the congestion rising between 07:30 and 09:00. A scatter plot shows the relationship between vehicle volume and speed. An accident-by-hour chart shows incident concentration. Weather information is displayed only if the selected dataset contains it.

The inspector switches to prediction. The separately developed ML component returns a +30 minute prediction. The map updates to show the predicted future congestion layer, clearly labeled as predicted rather than observed.

The system then identifies the location as a high-priority area and provides evidence: current severity, traffic volume, persistence, accident information, and predicted severity. The inspector uses this information for human decision-making.

This scenario is the reference demo narrative for the complete product.

30. Final Product Statement

The finished system should feel like a real Chennai traffic authority intelligence platform: **map-first, interactive, geospatial, multidimensional, explainable, and prediction-ready**. The product's strongest differentiator is not the number of charts; it is the ability to start from a geographic problem, click it, and progressively reveal the evidence behind that problem.